

Multi-Commodity Flow-Based Spreading in a Commercial Analytic Placer

Nima Karimpour Darav
Microsemi Corporation
Kitchener, Ontario, Canada
nima.karimpourdarav@microchip.com

Kristofer Vorwerk
Microsemi Corporation
Kitchener, Ontario, Canada
kris.vorwerk@microchip.com

Andrew Kennings
University of Waterloo
Waterloo, Ontario, Canada
akennings@uwaterloo.ca

Arun Kundu
Microsemi Corporation
San Jose, California, USA
arun.kundu@microchip.com

ABSTRACT

Modern analytic placement tools are commonly built around the idea of iterative Lower Bound (LB) and Upper Bound (UB) placement. The LB step optimizes wirelength and timing while ignoring overlap and cell-type constraints, whereas the UB step attempts to spread cells and satisfy constraints without harming design quality. Top-down geometric partitioning techniques have traditionally been used to spread cells during UB placement. We propose a new, network flow-based approach for UB placement which does a better job of preserving quality by optimizing the *displacement* of cells from their LB positions. Our approach not only addresses cell overlap, but also accommodates complex region constraints and simultaneously spreads unit-sized logic, carry chains, and blocks like RAMs and DSPs. Our technique is scalable, does not require geometric partitioning, and is suitable for both flat and clustered placement flows. We deployed our algorithm in a commercial FPGA CAD flow, and show that it reduces HPWL by 6.4% on average (up to 22.8% in the best case) while improving worst-slack timing in over 90% of designs, compared to a state-of-the-art alternative.

CCS CONCEPTS

• **Mathematics of computing** → **Network flows**; • **Hardware** → **Reconfigurable logic and FPGAs**; **Placement**.

KEYWORDS

FPGA placement; network flows; analytic placement

ACM Reference Format:

Nima Karimpour Darav, Andrew Kennings, Kristofer Vorwerk, and Arun Kundu. 2019. Multi-Commodity Flow-Based Spreading in a Commercial Analytic Placer. In *The 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '19), February 24–26, 2019, Seaside, CA, USA*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3289602.3293896>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FPGA '19, February 24–26, 2019, Seaside, CA, USA
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-6137-8/19/02...\$15.00
<https://doi.org/10.1145/3289602.3293896>

1 INTRODUCTION

Placement is a key component in the Computer-Aided Design (CAD) flow in that it accounts for a majority of the runtime while being largely responsible for overall design quality. Traditional placement algorithms based on simulated annealing [2] or min-cut partitioning [14] generally do not scale well, and this has led to a significant increase in interest for analytic placement techniques.

Many analytic placers are built upon the idea of iterative LB and UB placement. This strategy has been shown to produce competitive solutions in both the Application Specific Integrated Circuit (ASIC) [6, 11, 13] and Field Programmable Gate Array (FPGA) [9, 12, 15, 16] domains. Within the LB step, several objectives—such as wirelength and timing—are optimized while ignoring overlap constraints and other placement restrictions. The UB step seeks to produce a fairly non-overlapping placement with the goal of preserving the relative positions provided by the LB placement. To spread movable objects subject to defined constraints, the UB step in many modern ASIC and FPGA placement tools [6, 9, 11–13, 15, 16] exploits the idea of *rough legalization*. Full legalization and detailed improvement are applied to further enhance the quality and satisfy all remaining constraints.

For FPGA placement, the UB step must not only preserve the quality and information inherent in the LB placement, but it must also account for the presence of heterogeneous cells—Look-Up Tables (LUTs), Flip-Flops (FFs), different types of Random Access Memory (RAM), Digital Signal Processors (DSPs), and so forth—which are subject to varying placement constraints. To this end, we propose a new approach to UB placement which preserves the quality of the LB solution and accounts for the constraints imposed by modern FPGA architectures.¹

The main contributions of this paper are as follows:

- (1) Our technique empirically outperforms existing UB spreading heuristics [11, 13] in terms of quality while *maintaining comparable runtime*.
- (2) The proposed algorithm is suitable for spreading both flat and clustered FPGA netlists.

¹As an aside, our proposed UB placer was developed for FPGA placement within a commercial tool, but the idea of using flow for UB placement is equally applicable to ASIC placement in which heterogeneity is not as much of a concern.

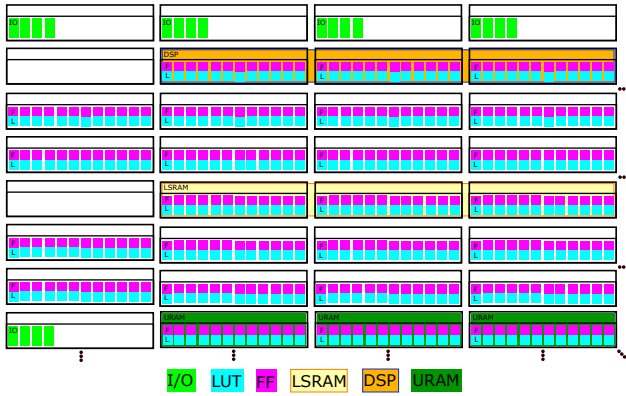


Figure 1: Logical view of the north-west corner of a Microsemi PolarFire FPGA. A similar pattern repeats throughout the die.

- (3) Our technique optimizes maximum and average cell displacement and preserves the relative ordering of cells from the LB solution.
- (4) All placeable objects are accounted for, during spreading, *at the same time*, even when those objects compete for the same resources.² Moreover, the technique considers the availability of FPGA resources and the impact to placement legality imposed by overlapping region constraints—a constraint not described by previous works.

The rest of this paper is organized as follows. Section 2 describes our target FPGA architecture and analytic placement flow, along with related work. Our proposed network flow-based spreading algorithm is elaborated in Section 3. Numerical results are presented in Section 4, where we demonstrate the superiority of flow-based spreading versus prior methods. Section 5 summarizes our work and provides directions for future investigations.

2 PRELIMINARIES

2.1 FPGA Architecture

We developed our tool for the Microsemi PolarFireTM FPGA architecture, a depiction of which is presented in Figure 1 [4]. It is a row-oriented FPGA consisting of I/Os along the periphery, 4-input LUTs, FFs, DSPs, URAMs, and LSRAMs blocks. Each 4-input LUT and FF are paired into *modules*. Groups of 12 modules are considered a *cluster*. Clusters have architectural constraints which must be observed when inserting modules into clusters (e.g., limits on control signals, clocks, and regular input and output signals). LUTs support *carry chains* for the efficient implementation of certain operations (e.g., counters and adders). LUTs involved in a carry chain *must* be placed adjacent to each other in the same row.

The URAM and LSRAM blocks support different RAM applications. Additionally, the architecture supports the use of *math chains*, consisting of multiple DSP blocks, to create wide multipliers. DSP

²Traditional flow-based methods such as bipartite matching handle neither mixed-sized movable objects nor large designs without dividing the design into smaller sub-problems [12]. In contrast, our proposed method handles mixed sized-objects such as *carry chains* and *math chains* seamlessly as part of the formulation.

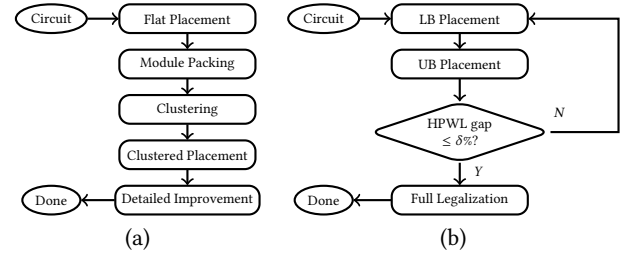


Figure 2: Typical CAD flow for FPGA placement in which different placements are obtained via analytic placement.

blocks involved in chains must be placed next to each other in the same row. In the absence of a DSP or RAM block, *LUTs and FFs can be placed in the interface logic sites normally occupied by the DSP and RAM to improve utilization, so long as those LUTs do not employ carry chains*. Further FPGA details are described in [4].

Different blocks can be different widths. If the width of a cluster is Ω , then DSP blocks have width 3Ω , URAM blocks have width Ω , and LSRAM blocks have width 3Ω . Carry chains and math chains which span multiple clusters and are an integer multiple of Ω . Objects smaller than a cluster (e.g., modules) have widths of $k\Omega$ where k is a fraction < 1 . The placement of a circuit into this FPGA requires the placement of unequally-sized, heterogeneous cells onto discrete physical locations. The architectural features of this FPGA are typical of all modern FPGAs.

2.2 CAD Flow

Microsemi's Libero[®] software implements a CAD flow similar to that illustrated in Figure 2 for placing PolarFire designs. In Figure 2(a), given a mapped design, LUTs and FFs are packed into modules which, in turn, are packed into clusters. Packing and clustering satisfy architectural constraints of the FPGA and ensure that the design will fit into the device while maximizing signal absorption in individual clusters [8]; it may be performed with or without an initial placement. An initial flat placement is not typically required to be a feasible placement—its purpose is to provide physical information to help the packing and clustering make better decisions about wirelength, timing and routability optimization. Subsequent to packing and clustering, clustered placement is performed to place all clusters as well as IP blocks (RAMs and DSPs) in the netlist. Detailed placement further optimizes the design by moving individual modules and instances between clusters.

Both flat and clustered placement can be performed via analytic placement as illustrated in Figure 2(b). LB placement is performed via quadratic programming using the Bound-to-Bound (B2B) model [19]. Weighted edges to target locations are included, along with additional weighted edges for timing optimization. UB placement performs a *rough legalization* to spread objects and find target locations for the subsequent LB placement. The Libero software has historically employed a variant of top-down geometric partitioning to spread cells in over-capacitated areas of the device while preserving the relative order of the movable objects from the LB solution. This geometric partitioning accounts for the heterogeneous nature of the FPGA netlists and device architectures.

Iterations continue until the HPWL of the LB and UB placements differ by $\leq \delta\%$. Legalization is applied to remove final overlap, to ensure that cells are placed onto suitable sites (depending on their type), and to verify that placement constraints are satisfied. Timing analysis and routability estimation (not shown in Figure 2) can be performed throughout analytic placement.

Unlike full legalization, which must ensure that all placement constraints are satisfied, the rough legalization performed during UB placement is not required to guarantee legality, in exchange for more efficient runtime. That said, it is desirable for the UB placer to produce a placement which is “as legal as possible” since the accuracy of the timing and routability estimation are improved when the spread-out placement closely matches the final placement.

2.3 Related work

We review several recent works with CAD flows similar to Figure 2. In [9], the HeAP wirelength-driven analytic placer is described for heterogeneous FPGA placement; it is a direct adaptation of the ASIC placer SimPL [11]. The HeAP placer is designed for clustered placement, with packing and clustering performed using a commercial tool. HeAP uses geometric partitioning and non-linear shifting [11] to spread movable objects in overfilled areas of the FPGA. Cut lines are placed during geometric partitioning to account for the architecture, and the placement of heterogeneous blocks is accomplished by spreading different block types separately.

Several analytic placers for FPGAs were introduced in [12, 15, 16] after the International Symposium on Physical Design (ISPD) placement contests of 2016 and 2017 [17, 18]. In [15], the GPlace-pack and GPlace-flat placers are described, wherein UB placement is performed using top-down geometric partitioning. The tool RippleFPGA was introduced in [16]—based on the work of [10]—in which top-down geometric partitioning and FPGA constraints are taken into account during the legalization. UTPlacerF [12] performs both flat and clustered placement and closely follows the flow in Figure 2, with its UB placement based on SimPL. During flat placement, both a wirelength-driven and routability-driven phase are employed, but both phases use analytic placement as shown in Figure 2(b). RAM and DSP blocks are legalized using bipartite matching. As stated in [12], solving a complete matching problem is impractical for larger designs and, consequently, the placement region is divided into a set of uniform rectangles and matching is applied to each rectangle separately. After flat placement, packing and clustering are performed using physical information, and clustered placement is computed with an analytic method.

For rough legalization, the aforementioned approaches mainly rely on heuristics and top-down geometric partitioning (augmented with other heuristics to handle heterogeneity). These methods may not be able to preserve the quality of the LB placement since the minimum perturbation during the process is not guaranteed.

The spreading problem was identified as an important step worthy of its own investigation in [20], where the use of top-down geometric partitioning was questioned. The approach presented in [20] does not rely on partitioning, but rather on creating a force map akin to that used in ASIC placement tools (e.g., [7]). The flow-based strategy which we describe in this work spreads all block types simultaneously whereas [20] only spreads logic blocks and

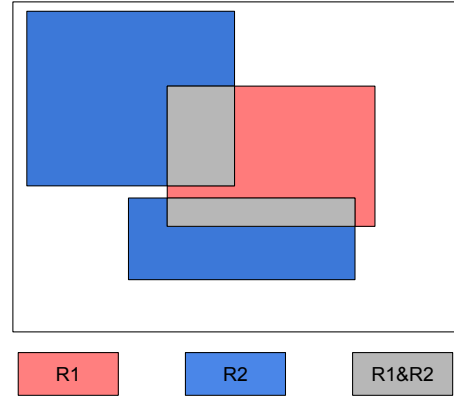


Figure 3: Two overlapping region constraints R1 (in pink) and R2 (in blue). The overlapping area is highlighted in gray.

requires that RAM and DSP blocks be spread separately. Our technique is runtime-efficient and scalable, and accounts for both maximum and average cell displacement, which preserves the quality of the LB placement while maintaining the ordering of movable objects. Finally, our flow-based algorithm handles overlapping region constraints which can restrict the placement of objects within the FPGA—a constraint not considered in previous works.

2.4 Region Constraints

Region constraints may be specified in the Libero software flow as part of a user-initiated floorplanning strategy [3]. Several overlapping region constraints can be defined for a design, as shown in Figure 3. If a cell is assigned to a region R , the cell must be placed inside the region; however, other clusters not assigned to any region can be placed either inside or outside of R . In contrast to the constraints considered in [6], the region constraints in Libero may be *overlapping*—this poses a *significant* challenge because cells may compete for the same locations, so all regions must be accounted for simultaneously.

3 PROPOSED SPREADING ALGORITHM

Our proposed spreading algorithm is inspired by [5] which is a legalization tool for ASICs. However, there are key differences for FPGA placement and for spreading. We begin by defining some nomenclature. Let D represent the set of logical (placeable) clusters in a design, comprising 5 subsets: D_a , D_m , D_u , D_s , and D_l , corresponding to the carry chains, DSP, URAM, LSRAM, and general logic clusters (i.e., no carry chains), respectively.³ Let n be a positive integer and $\Omega = 12$ which is the maximum number of modules in a physical cluster (cf. Section 2). We define the width (W) for each logical cluster in each subset as:⁴

- $\forall d_{a,i} \in D_a, W(d_{a,i}) = n\Omega$. Thus, logical cluster $d_{a,i}$ (a carry chain) may horizontally span several physical clusters.

³I/Os in commercial FPGAs can be subject to special placement and timing considerations, and the details of their automatic placement are beyond this paper.

⁴Although we present this work for a clustered placement flow, we note that our proposed method can be directly used for a flat placement flow by setting Ω to an appropriate value and defining, for example, an additional set D for modules.

- $\forall d_{m,i} \in D_m, W(d_{m,i}) = 3n\Omega$. Thus, logical cluster $d_{m,i}$ (a single DSP or a math chain) will horizontally span several physical clusters.
- $\forall d_{u,i} \in D_u, W(d_{u,i}) = \Omega$.
- $\forall d_{l,i} \in D_l, W(d_{l,i}) = \Omega$.
- $\forall d_{s,i} \in D_s, W(d_{s,i}) = 3\Omega$.

Similar to [6], any defined region constraint is tagged by a number (color) $r > 0$ and the die area is tagged (colorized) by $r = 0$ as the *default region*. Let bins V represent the nodes of a flow network, composed of 4 subsets: V_a, V_m, V_u , and V_s , corresponding to the carry chain sites, DSP sites, URAM sites, LSRAM sites of the FPGA architecture, respectively, which are vertically aligned with the sites of the FPGA architecture. Each bin $v_i \in V$ has the same height as an FPGA site but with the width equal to Ω . Therefore, each DSP or LSRAM site is represented by three bins because the width of a DSP or LSRAM site is equal to 3Ω . Each bin v_i keeps a color vector of regions overlapping with bin v_i . Let objects Γ represent part of the logical cluster set D in our flow network, where each object $\gamma_i \in \Gamma$ has the same size as a bin $v_i \in V$. (For flat placement, the size of an object would be smaller than a bin.) Objects representing a cluster have the same color as the region the cluster is assigned to. Let the width of an object γ_i be $W(\gamma_i)$. The total width of objects $\Gamma(v_i)$ in a bin v_i is computed as $W(\Gamma(v_i)) = \sum_{\gamma_i \in \Gamma(v_i)} W(\gamma_i)$. Let $supply(v_i) = \max\{0, W(\Gamma(v_i)) - \Omega\}$. A bin v_i is an overflowed (supply) bin if $supply(v_i) > 0$; or, put differently, if the total width of objects $\Gamma(v_i)$ assigned to bin v_i is greater than the width of bin v_i , bin v_i is considered to be overflowed because each bin cannot include more than one object without overlap.

Let $demand(v_i) = \max\{0, \Omega - W(\Gamma(v_i))\}$. A bin v_i is a sink bin if $demand(v_i) > 0$. (Sink bins do not include any objects.) Resolving all overflowed bins is the main objective of our algorithm subject to several constraints that are described further in this paper. The problem is considered a *multi-commodity* flow problem [1] for two key reasons. First, the set Γ is composed of five subsets $\Gamma_a, \Gamma_m, \Gamma_u, \Gamma_s$, and Γ_l which in turn represent logical clusters D_a, D_m, D_u, D_s , and D_l . The first four subsets of objects can be transferred only between two bins in the same subset while the objects of set Γ_l can be transferred between any two bins. For example, if bin $v_{a,1} \in V_a$ includes two objects $\gamma_{a,1} \in \Gamma_a$ and $\gamma_{l,1} \in \Gamma_l$, $\gamma_{a,1}$ may be transferred only to a bin in subset V_a while $\gamma_{l,1}$ can be transferred to any given bin. Second, there may be several objects which together represent a wide logical cluster, subject to the constraint that those objects should be placed at bins that are on the same row and horizontally-adjacent. A bin v_i is *compatible* with an object γ_j if and only if either the color of object γ_j is zero (representing *default region*) or the color of object γ_j is included in the color vector of bin v_i and either $\gamma_j \in \Gamma_l$ or $\gamma_j \in \Gamma_x$ and $v_i \in V_x$ where $x \in a, m, u, s$. In our approach, heterogeneous clusters are handled seamlessly by using a single network flow problem.⁵ A summary of terms used in this paper is presented in Table 1.

Table 1: Nomenclature used in this paper.

Term	Description
n	A positive integer.
Ω	The maximum number of modules in a physical cluster.
A	The given FPGA architecture.
N	The given circuit.
D	The set of placeable clusters in the design.
D_x	The set of clusters with the same type. D_a, D_m, D_u, D_s , and D_l in turn indicate the set of carry chains, DSP, URAM, SRAM, and general logic clusters.
$d_{x,i}$	Cluster i in set D_x .
V	The set of nodes (bins) of the network flow constructed for the design.
$ V $	The size of set V .
V_x	The set of bins with the same type. V_a, V_m, V_u , and V_s , and V_l in turn indicate the set of carry chain sites, DSP sites, URAM sites, SRAM sites.
$v_{x,i}$	Bin i in set V_x .
Γ	The set of network flow objects representing clusters D .
$ \Gamma $	The size of set Γ .
Γ_x	The set of network flow objects with the same type. $\Gamma_a, \Gamma_m, \Gamma_u, \Gamma_s$, and Γ_l represent D_a, D_m, D_u, D_s , and D_l , respectively.
$\gamma_{x,i}$	Object i in set Γ_x .
$\Gamma(v_i)$	The set of objects inside bin v_i .
$\hat{\Gamma}(d_i)$	The set of objects representing cluster d_i .
$W(d_i)$	The width of cluster d_i .
$W(\gamma_i)$	The width of an object γ_i .
$W(\Gamma(v_i))$	The total width of objects assigned to bin v_i .
$demand(v_i)$	The demand value of a bin v_i .
$supply(v_i)$	The supply value of a bin v_i .
r	The region tagged with r . $r = 0$ is used for the default region (the die area).
Λ	The set of overfilled bins at iteration $iter$.
$ \Lambda $	The size of set Λ .
λ_i	An overfilled bin ($supply(\lambda_i) > 0$).
$iter$	The current iteration of the algorithm.
Ψ	The upper-bound for maximum object movement at the current iteration of the algorithm.
$P(\lambda_i)$	The set of augmenting paths identified by Algorithm 2 for overfilled bin λ_i .
$ P _{avg}$	The average number of the identified paths at each iteration.
p_k	An augmenting path in $P(\lambda_i)$
$demand_T$	The total demand provided by augmenting paths $P(\lambda_i)$ identified by Algorithm 2.
$\psi(\gamma_j, v_i)$	The quadratic displacement of moving object γ_j to bin v_i .
$wt(\gamma_i)$	The weight of object γ_i .

⁵Our proposed algorithm can be easily adapted to address routability based on cell bloating or target density. For example, a cell bloating technique [12] is accomplished by changing object widths. For target density adjustment [16], bin capacities can be adjusted.

Algorithm 1: Network-Flow Based Spreading Algorithm

Require: For the given FPGA architecture A , the circuit N has been packed into a set of clusters D .

- 1: $\Gamma = \text{createObjectsFromClusters}(D)$
- 2: $V = \text{extractBins}(A)$
- 3: $E = \text{connectBins}(V)$
- 4: $\text{assignObjectsToBins}(\Gamma, V)$
- 5: $iter = 0$
- 6: **repeat**
- 7: $\Psi = \text{computeMaxMovement}(iter)$
- 8: $\Lambda = \text{identifyOverflowedBins}(V)$
- 9: Sort bins Λ in order of ascending supply values
- 10: **for** each bin $\lambda_i \in \Lambda$ **do**
- 11: $P(\lambda_i) = \text{identify candidate paths using Algorithm 2}$
- 12: Sort paths $P(\lambda_i)$ in order of ascending path costs
- 13: **for** each $p_k \in P(\lambda_i)$ **do**
- 14: **if** $\text{supply}(\lambda_i) > 0$ **then**
- 15: Move cells over path p_k (see Section 3.3)
- 16: **end if**
- 17: **end for**
- 18: **end for**
- 19: $iter = iter + 1$
- 20: **until** Λ is empty
- 21: $\text{updateClusterPositions}(D, \Gamma, A, V)$

3.1 Algorithm Outline

The outline of our algorithm is presented in Algorithm 1. First, a set Γ of objects is created based on the given set D of clusters (line 1). For a given logical cluster d_i , if $W(d_i) = n\Omega$ (where n is a positive integer number), a set $\hat{\Gamma}(d_i)$ including n objects will be created such that the width of each object is equal to Ω . The color of objects representing a cluster is determined based on the color of the region the cluster is assigned to. In line 2, a set V of bins are extracted for the given FPGA device A such that each bin v_i includes a color vector of all regions (including *default region*) overlapping with bin v_i . Then, bins V are connected to each other in four directions for each region a bin is overlapping with: left, right, top, and bottom. In contrast to [5], each bin v_i is vertically connected to multiple bins in different subsets to allow objects from different subsets to be easily transformed to *compatible* bins. For example, if a given bin $v_{a,1} \in V_a$ includes two objects $\gamma_{l,1} \in \Gamma_l$ and $\gamma_{a,1} \in \Gamma_a$, additional edges allow object $\gamma_{a,1}$ to be transferred to the closest bin $v_{a,2} \in V_a$ on top or the closest bin $v_{a,3} \in V_a$ on bottom. Similarly, there are extra edges added to allow $\gamma_{l,1}$ to be vertically transferred to any closest bin in set V .

Figure 4a shows how a typical bin is connected. Since all bins at the same row are in the same subset, a bin v_i is horizontally-connected to only the two closest bins on the left and right sides. If v_i was overlapping with any defined region constraint $r > 0$, extra edges would be added to connect bins (with different types for four directions) overlapping with region constraint r .

This method for connecting bins prevents deadlocks which may be caused by blockages, overlapping regions, or a highly-utilized subset of bins. More importantly, both this method and the definition of *compatible bin* are key to the ability of our proposed method to: (1) spread all types of logic in all defined region constraints simultaneously; and, (2) resolve the placement of different types

of objects onto spots in the FPGA device which can accommodate different types of blocks.

In line 4, each object γ_i representing a cluster d_i is assigned to a *compatible* bin v_i with the closest quadratic distance from cluster d_i . In lines 6-20, an iterative process is applied to compute the maximum displacement and to resolve all overflowed bins. In line 7, the upper bound for maximum object movement is relaxed at each iteration based on the quadratic equation: $\Psi = ((iter + 1) \cdot \Omega)^2$, where $iter$ indicates the number of the current iteration.⁶ In practice, the quality of the final solution is mostly independent of the order of resolving overflowed bins because cell movements are limited by Ψ ; however, sorting overflowed bins Λ in order of ascending supply values has been shown to reduce runtime. This is because resolving slightly-overflowed bins first provides more candidate paths for highly-overflowed bins later—see Figure 4b and Figure 4c.

In lines 10-18, for each overflowed bin λ_i , a set $P(\lambda_i)$ of augmenting paths are identified, set $P(\lambda_i)$ is sorted in order of ascending cost to ensure that cells are moved with the minimum cost, and objects are moved along each path $p_k \in P(\lambda_i)$. Finally, the location of each logical cluster $d_i \in D$ is updated based on the location (central gravity) of its representing object (objects). Hence, the solution provided by Algorithm 1 is well-spread as clusters represented by single objects are fully-legalized while minor overlaps are permitted for clusters represented by multiple objects.

3.2 Augmenting Path Algorithm

We propose a variant of Breadth-First Search (BFS) to identify multiple candidate paths satisfying the following three constraints: (1) the quadratic displacement of any object movement along the identified paths is $\leq \Psi$; (2) there is at least one object in each bin that can be moved to the next bin along the identified paths while satisfying the first constraint; and, (3) the last bin in a candidate path is empty (i.e., a sink bin). The first constraint prevents unnecessarily long displacement. The second constraint avoids paths leading to a deadlock. The third constraint guarantees that a portion of the supply of the overflowed bin can be sunk along the identified paths.

Algorithm 2 begins by initializing the total capacity of all identified paths, demand_T , to zero, and initializing the “visited” status of each bin; each bin cannot be traversed more than once. The overflowed bin λ_i is inserted to an empty path p in line 6. Each path is a queue of bins whose first element is the overflowed bin λ_i . While traversing bins using our variant of BFS, any partial path is saved into a queue Q . Lines 8-28 are repeated until queue Q is empty or the total demand (demand_T) provided by the identified paths can sink supply λ_i . The last bin v_{src} of path p on the head of queue Q is determined; then, for each bin v_i connected to bin v_{src} , a partial path p_i is created if there is at least one *compatible* object in v_{src} that can be moved to bin v_i (lines 11-27).

The minimum cost of moving objects from v_{src} to v_i is computed using the equation: $\text{cost} = \text{wt}(\gamma^*) \cdot \psi(\gamma^*, v_i)$, where γ^* is an object in v_{src} with the minimum cost such that the object is *compatible* with v_i and the quadratic movement $\psi(\gamma^*, v_i)$ of object γ^* is less than the upper bound maximum movement Ψ at the current iteration of Algorithm 1. If there is no such object in v_{src} , cost is set to ∞ .

⁶In [5], a linear equation is used for maximum movement, but we have empirically found that a quadratic approach is better for *spreading*.

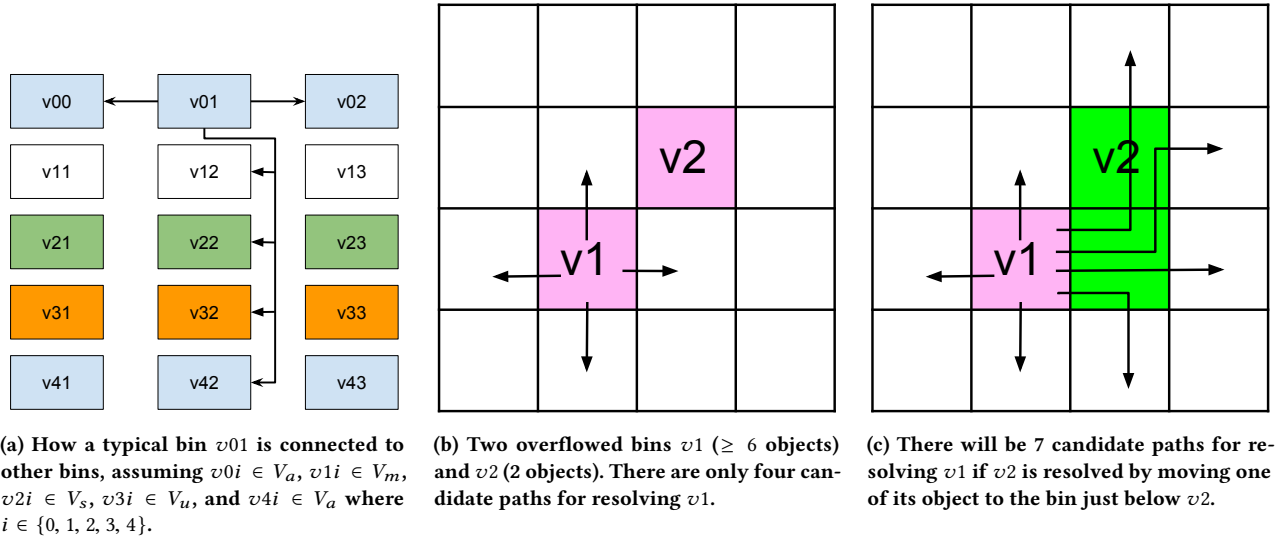


Figure 4: Illustrative examples of (a) connections between bins, (b) two overflowed bins, and (c) the increase in the number of candidate paths.

Algorithm 2: Multiple Augmenting Paths

Require: Set V of all bins, set E of all network flow edges, overflowed bin λ_i , and maximum movement Ψ are given.

```

1:  $demand_T = 0$ 
2: for each  $v_i \in V$  do
3:    $visited(v_i) = \text{false}$ 
4: end for
5:  $visited(\lambda_i) = \text{true}$ 
6: Insert  $\lambda_i$  into an empty path  $p$ 
7: Insert path  $p$  into an empty queue  $Q$ 
8: repeat
9:   Dequeue path  $p$  from queue  $Q$ 
10:   $v_{src} = \text{the tail of path } p$ 
11:  for each bin  $v_i$  connected to bin  $v_{src}$  do
12:    if  $visited(v_i) \neq \text{true}$  then
13:       $cost = \text{computeCost}(v_{src}, v_i, \Psi)$ 
14:      if  $cost \neq \infty$  then
15:        Create a copy  $p_i$  of path  $p$ 
16:         $cost_T(p_i) = cost_T(p) + cost$ 
17:        Insert bin  $v_i$  into path  $p_i$ 
18:         $visited(v_i) = \text{true}$ 
19:        if  $|\Gamma(v_i)| == 0$  then
20:          Insert  $p_i$  into set  $P(\lambda_i)$ 
21:           $demand_T = demand_T + \Omega$ 
22:        else
23:          Insert path  $p_i$  into queue  $Q$ 
24:        end if
25:      end if
26:    end if
27:  end for
28: until  $Q$  is empty or  $supply(\lambda_i) \leq demand_T$ 

```

The function $wt(\gamma^*)$ computes a weight based on the size of set $\hat{\Gamma}(d_i)$ representing logical cluster d_i where $\gamma^* \in \hat{\Gamma}(d_i)$. The term $wt(\gamma^*)$ plays an important role in our multi-commodity flow model

because it maintains the objects representing a single cluster close to each other. In lines 19-24, if the computed cost is a valid number, the new path p_i either will be considered a candidate path if v_i is empty or will be inserted into Q for further investigation.

3.3 Object Movement

Objects are moved along a path p using Algorithm 3. Before moving objects along a candidate path p , the cost of movement is recomputed as it might have been changed by moving objects along other candidate paths sharing some partial paths with path p . In addition, the order of traversing the path is reversed using a stack to avoid moving an object along path p several times. To achieve the minimum total movement, before moving an object between two subsequent bins v_{src} and v_{sink} , objects in v_{src} are sorted in order of distance to v_{sink} . Figure 5 shows an example of how objects may move along a candidate path. In this figure, v_1 and v_2 are overflowed bins with 2 objects each, while v_3 is a sink bin. Bins v_1 and v_3 are overlapping with region R_2 in Figure 3 while v_2 is overlapping with both regions R_1 and R_2 . First, the closest object to bin v_3 is moved to sink bin v_3 (Figure 5a) such that the object must be *compatible* with the target bin v_3 . c_3 is not *compatible* with bin v_3 because c_3 is assigned to region R_1 while v_3 is overlapping with only region R_2 . Then, the closest *compatible* object to bin v_2 is moved to v_2 . As shown in Figure 5c, no new overflowed bin is introduced and the source bin (v_1) in the augmentation path is no longer overflowed.

4 EXPERIMENTAL RESULTS

Our flow-based spreading algorithm was implemented in Microsemi's Libero tool. The quality of our technique was compared to Microsemi's previous UB heuristic which employed top-down geometric partitioning and bipartite matching [9, 12]. Since the detailed

Algorithm 3: Cell Move

Require: Set V of all bins, set E of all network flow edges, path p and the maximum movement Ψ are given.

- 1: Dequeue bin v_{src} from path p
- 2: Insert bin v_{src} into a stack S
- 3: **while** p is not empty **do**
- 4: Dequeue bin v_{sink} from path p
- 5: $cost = computeCost(v_{src}, v_{sink}, \Psi)$
- 6: **if** $cost == \infty$ **then**
- 7: **return**
- 8: **end if**
- 9: $v_{src} = v_{sink}$
- 10: Insert bin v_{sink} into stack S
- 11: **end while**
- 12: Pop bin v_{sink} from stack S
- 13: **while** S is not empty **do**
- 14: Pop bin v_{src} from stack S
- 15: Sort cells $\Gamma(v_{src})$ in bin v_{src} in order of the closest to bin v_{sink}
- 16: Move the first *compatible* object from set $\Gamma(v_{src})$ to v_{sink}
- 17: $v_{sink} = v_{src}$
- 18: **end while**

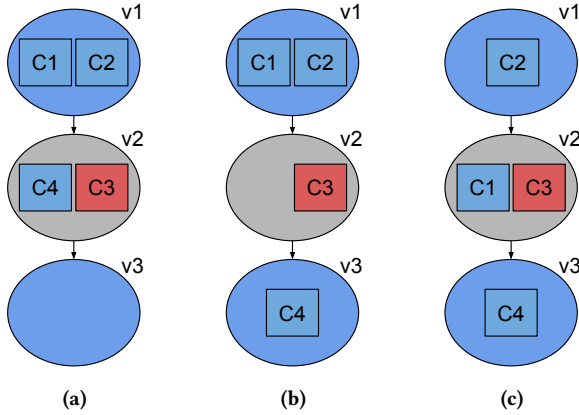


Figure 5: An example of how objects are moved along a path.

improvement step—cf. Figure 2(a)—can make additional improvements to the solution, the results in this paper are reported for the legalized placements *just after* the analytic placement step (without applying detailed improvement).

A benchmark suite of 557 designs targeting the PolarFire family was employed. The designs originate from both industry as well as Microsemi’s internal development, and contain a broad mixture of logic, RAM, DSP, high-speed I/O interfaces, complex local and global clocking regimes, complicated region constraints, timing constraints, carry chains, and varying logic depths. All designs used in this work contain at least 5000 placeable logic cells, with some designs containing over 700k. The timing constraints are adjusted so that there are no critical clock-to-input or clock-to-output timing constraints (which would render the timing of each design largely dependent on I/O placement, as opposed to internal logic). Each design is placed into the smallest available PolarFire die which could contain it, as this ensures that the utilization presents

a sufficient challenge. The layout tools are configured to optimize as best as possible and not stop early if timing is met. Since the initial random seed can alter the “starting point” for the packing and clustering algorithms implemented in the commercial tool, each design is executed with 5 different random seeds and the average value of each numerical result is reported in this paper.

4.1 HPWL and Worst Slack Evaluation

Figure 6 shows the per-design improvement in Half Perimeter Wire Length (HPWL) which was achieved by switching from the original UB algorithm to our proposed flow-based algorithm, across the suite of 557 designs. Designs are sorted on the x -axis based on the achieved improvement in HPWL. The right y -axis and the left y -axis respectively show the size of a design and the percentage of HPWL improvement achieved by switching from the original UB algorithm to our proposed flow-based algorithm.

The improvement in HPWL due to flow-based spreading is 6.4% on average, with very few degradations. In some cases, the HPWL of the placement was improved by up to 22.8%. The approximate size of each design (in terms of the number of logic cells) is plotted as a line graph along the secondary axis—this shows that the improvement is not confined to any particular design size.

We examined the worst-slack estimates at the end of analytic placement as a means of evaluating our proposed algorithm’s impact on design timing. Across the design suite, over 90% of the designs showed either equivalent or better critical path slack.⁷ The average improvement in estimated worst slack was 389 ps per design which, in the PolarFire architecture, is approximately the same as the delay through six LUT ($A \rightarrow Y$) arcs. It is important to note that the flow-based spreading does not attempt to explicitly optimize HPWL or slack, but rather, achieves these improvements due to its superior ability in minimally perturbing the placement and object ordering.

4.2 Time Complexity and Runtime Analysis

Determining the time complexity of the proposed algorithm is challenging since the complexity is not only dependent on the number of bins and overfilled bins, but also the average number of paths identified at each iteration, which is itself a function of the number of sink bins.

The number of iterations in Algorithm 1 is bounded by $O(|v|^{0.5})$ as when $iter = O(|V|^{0.5})$, Ψ is set to a value such that any object displacement inside the die area is not greater than Ψ , and the remaining overfilled bins are completely resolved. Given the fact that the design utilization cannot be more than 100%, the number of objects cannot be greater than the number of bins ($|\Gamma| \leq |V|$). For each overfilled bin, the search space in Algorithm 2 is bounded from above and the upper-bound is $O(|V|)$ as the number of connections between a bin to other bins is limited and constant (see Figure 4a); however, the time complexity of lines 13-17 of Algorithm 1 will be on the order of $O(|V| \cdot |P|_{avg})$, ignoring the sort function in Algorithm 3.⁸ Therefore, the upper-bound of the time complexity

⁷The improvement in timing is presented in terms of placement-based timing estimation since the results were collected prior to routing.

⁸It is fair to ignore the sort function in Algorithm 3 if $|\Lambda|$ is large enough since the average number of objects in overfilled bins would be limited by a constant number.

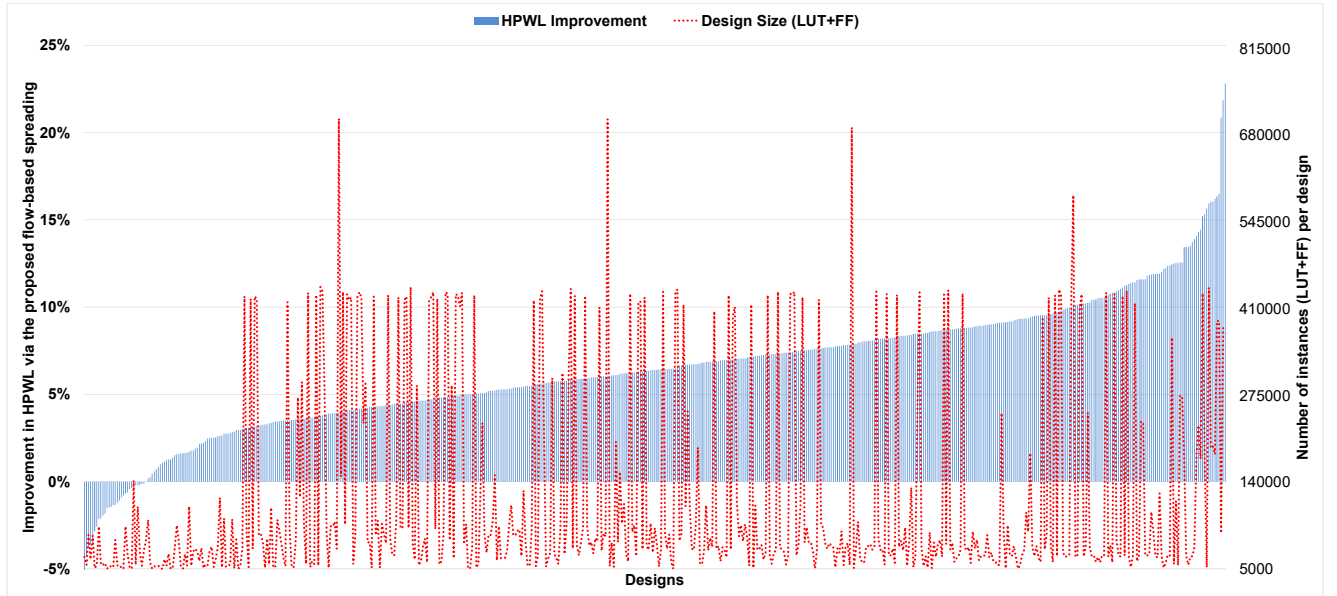


Figure 6: Improvement in HPWL due to flow-based UB spreading relative to Microsemi’s original UB placer. Each blue bar represents the improvement in legalized HPWL for a design in the 557-design suite (averaged over 5 seeds), with design size plotted in red against the secondary axis.

of the entire algorithm will be $T = O(|V| \cdot \min(|V|^{0.5}, |\Lambda|) \cdot |P|_{avg})$ as several bins are resolved at each iteration. Since $|\Lambda| \leq \frac{|V|}{2}$, $|V|^{0.5} < \frac{|V|}{2}$ for $|V| > 4$, and $|P|_{avg} = |V|$ for the worst case, $T = O(|V|^{2.5})$.

In practice, the flow-based spreading is very fast, particularly in the latter phases of placement when there is less overlap. Furthermore, given the higher quality of solution produced by an analytic placer with our flow technique, less time is required in latter steps of the CAD flow, such as detailed improvement and routing.

For illustrative purposes, we elaborate on the runtime of a subset of our design suite in Table 2. This subset consists of 20 designs chosen from among the suite of 557 designs based primarily on utilization. Within this sub-suite of designs, our proposed method achieved an average improvement of 6.57% in HPWL, and an improvement of 448 ps in worst slack. These results are achieved with just a 1.13% increase in runtime to the analytic placement. In our experience, this runtime increase was easily recovered from the reduced effort in subsequent phases of the CAD flow.

4.3 Displacement Investigation

The improvement in quality which was achieved by flow-based spreading can be explained predominantly by the reduction in average and maximum displacements at each iteration of the analytic placement. Figure 7 presents the improvement in average and maximum displacements at each iteration of analytic placement for a medium-sized, industrial design in the benchmark suite. Similar improvement has been observed across most other designs.

To further investigate the amount of displacement during UB placement, we selected two LB placements and subjected them to both our proposed flow-based and the original partitioning-based UB placement steps. Figure 8 illustrates the results for an early iteration of the analytic placement and the end of the analytic placement.

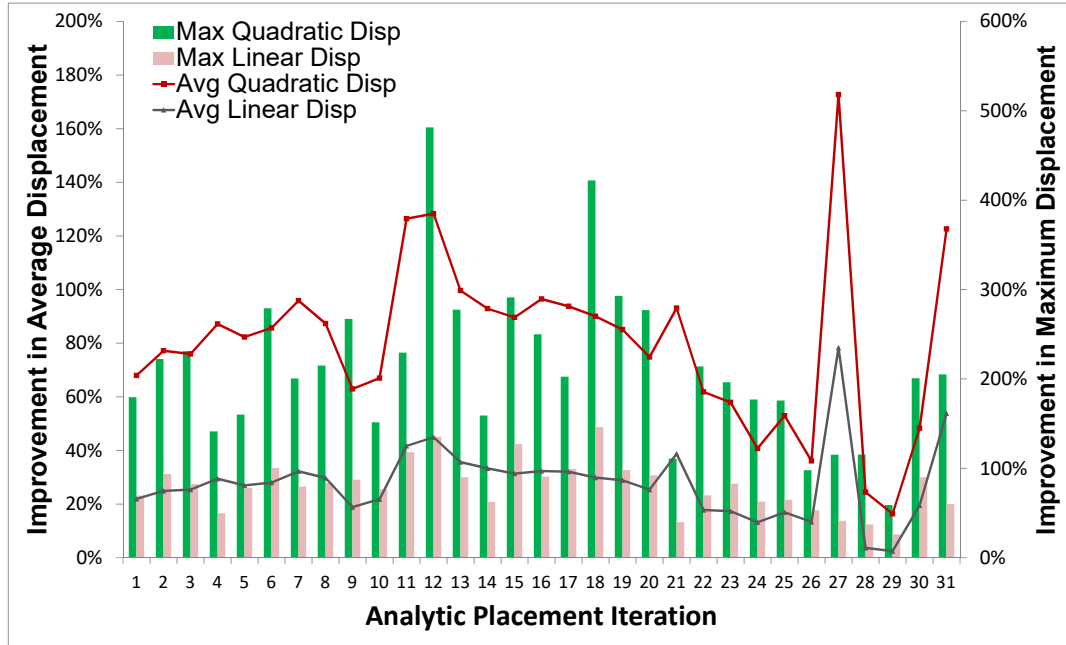
The displacement histograms in Figure 8 show that, regardless of the placement iteration (equivalently, the amount of overlap in the LB placement), our proposed flow-based spreading is capable of spreading more cells with less displacement compared to the original algorithm. Moreover, spreading is achieved with significantly less maximum displacement. The reduction in movement from our proposed flow-based spreading is more pronounced in early placement iterations where significantly more effort (movement) is required to purge overlap from the LB placement. It is remarkable that such improvement is possible near the end of placement when displacements are expected to be small. Although Figure 8 is for only one medium-sized design, the same trends appear across most designs.

5 CONCLUSIONS

We presented a novel network flow-based spreading algorithm for use in analytic placement, with several advantages compared to traditional methods such as [11, 13]. Our spreading algorithm is capable of handling heterogeneous blocks and device resources simultaneously. Additionally, it can handle complicated, overlapping region constraints which are not addressed in published academic works, but which must be considered for commercial deployment.

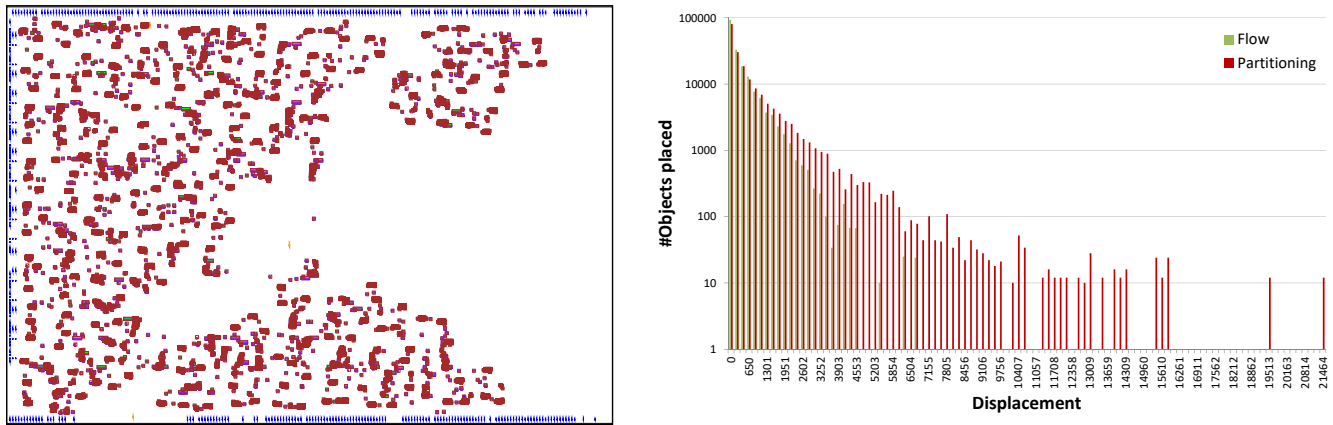
Table 2: Comparison of HPWL and slacks on 14 designs chosen from the design suite.

Design	Number of Blocks				Utilization (%)				Current Analytic Placer			Flow-Based Spreading		
	LUT+FF	URAM	LSRAM	DSP	LUT+FF	URAM	LSRAM	DSP	HPWL 10 ⁵	WS (ns)	Runtime (min)	HPWL 10 ⁵	WS (ns)	Runtime (min)
Test1	704912	3442	1296	780	73.2	77.5	85.3	52.7	69.69	-17.868	42.0	67.66	-13.369	46.0
Test2	433865	400	934	0	72.4	14.4	98.1	0.0	17.20	-3.264	18.5	16.31	-4.815	18.7
Test3	692105	3358	1293	775	71.9	75.6	85.1	52.4	54.90	-12.340	41.1	51.18	-11.089	44.4
Test4	103896	280	12	0	17.3	10.0	3.4	0.0	3.32	-0.666	3.8	3.12	-0.493	3.8
Test5	435750	1460	808	785	72.7	52.7	84.9	85.0	43.45	4.891	21.8	42.50	5.117	24.3
Test6	82415	96	0	0	13.8	3.5	0.0	0.0	4.59	-2.191	3.7	4.50	-2.903	3.6
Test7	232099	256	112	0	38.7	9.2	11.8	0.0	7.67	-1.035	10.8	7.14	-0.691	10.3
Test8	102179	0	0.0	924	17.1	0.0	0.0	100.0	3.36	-3.590	5.2	3.36	-3.387	5.4
Test9	434927	1484	806	784	72.6	53.5	84.7	84.9	28.48	4.371	22.2	25.46	5.039	22.2
Test10	122589	2772	0	0	20.5	100.0	0.0	0.0	6.57	-14.798	6.3	6.49	-15.098	6.3
Test11	195778	341	861	3	32.7	12.3	90.4	0.3	8.98	-1.647	6.3	8.56	-0.894	6.5
Test12	274064	304	640	0	45.7	11.0	67.2	0.0	12.74	-1.566	11.0	11.78	-0.912	10.7
Test13	49291	0	0	0	22.7	0.0	0.0	0.0	1.46	2.701	2.1	1.37	2.771	2.1
Test14	424168	1475	783	777	70.8	53.2	82.3	84.1	34.91	5.141	21.3	32.53	2.039	21.9
Test15	93539	0	0	0	43.1	0.0	0.0	0.0	1.82	-1.985	4.2	1.65	-1.201	3.9
Test16	183402	0	0	100	47.7	0.0	0.0	17.0	7.94	-9.135	11.0	6.63	-6.296	10.7
Test17	430012	1428	806	784	71.8	51.5	84.7	84.9	39.64	4.835	22.4	37.67	4.837	25.2
Test18	393376	0	0	0	65.7	0.0	0.0	0.0	19.92	-2.298	19.4	17.73	-0.737	17.6
Test19	431926	1474	809	769	72.1	53.2	85.0	83.2	32.91	5.077	22.5	30.31	4.720	23.6
Test20	70550	9	68	7	32.5	0.9	19.3	2.1	3.27	-6.756	4.5	3.20	-5.799	4.7
Average Improvement												6.57%	+448 ps	-1.13 %

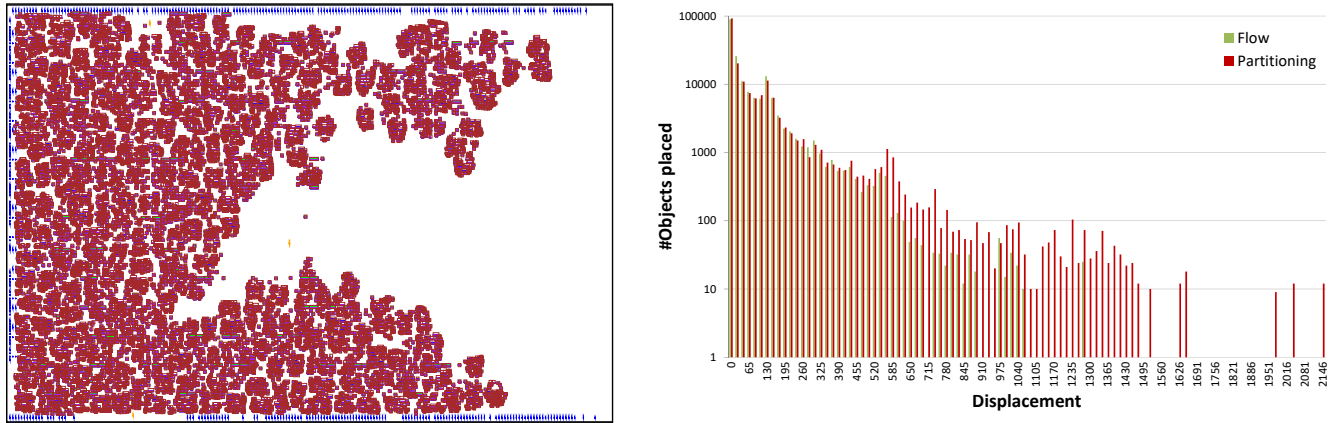
**Figure 7: Improvement in displacement at each iteration of analytic placement due to our flow-based UB spreading, compared to Microsemi's original UB spreading, for a medium-sized industrial design.**

By incorporating our algorithm into a commercial CAD flow, we showed an average improvement of 6.4% in HPWL and 389 ps in estimated worst-case path timing, after analytic placement, across a large set of industrial designs. Our improvements were achieved

with comparable runtime and without changing any other steps of the CAD flow. We attribute the success of our strategy to its ability to optimize the perturbation from a LB placement.



(a) LB placement and its displacement histograms near the start of analytic placement.



(b) LB placement and its displacement histograms near the end of analytic placement. Note that the scale of the displacement axis is much smaller, near the end of placement, because cells do not need to move as far in order to be “well-spread”.

Figure 8: The average and maximum displacements are significantly reduced when using flow-based spreading compared to Microsemi’s original UB spreading technique, at all stages of the analytic placement process.

REFERENCES

- [1] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin. 1993. *Network Flows: Theory, Algorithms, and Applications*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA.
- [2] V. Betz and J. Rose. 1997. VPR: A new packing, placement and routing tool for FPGA research. In *Proc. FPL*. 213–222.
- [3] R.C. Cofer and B.F. Harding. 2006. *Rapid System Prototyping with FPGAs: Accelerating the Design Process*. Elsevier Science. 146–147 pages.
- [4] Microsemi Corporation. 2017. *PO0137 Product Overview PolarFire FPGA*. Microsemi Corp., San Jose, CA.
- [5] N. K. Darav, I. S. Bustany, A. Kennings, D. Westwick, and L. Behjat. 2018. Eh?Legalizer: A High Performance Standard-Cell Legalizer Observing Technology Constraints. *TODAES* 23, 4 (2018), 43:1–43:25.
- [6] N. K. Darav, A. Kennings, A. F. Tabrizi, D. Westwick, and L. Behjat. 2016. Eh?Placer: A high-performance modern technology-driven Placer. *TODAES* 21, 3 (April 2016), 37:1–37:27.
- [7] Hans Eisenmann and Frank M. Johannes. 1998. Generic Global Placement and Floorplanning. In *Proc. DAC*. 269–274.
- [8] W. Feng, J. Greene, K. Vorwerk, V. Pevzner, and A. Kundu. 2014. Rent’s rule based FPGA packing for routability optimization. In *Proc. FPGA*. 31–34.
- [9] M. Gort and J. H. Anderson. 2012. Analytic placement for heterogeneous FPGAs. In *Proc. FPL*. 143–150.
- [10] X. He, T. Huang, L. Xiao, H. Tian, and E. F. Y. Young. 2013. Ripple: A robust and effective routability-driven placer. *TCAD* 32, 10 (October 2013), 1546–1556.
- [11] M. C. Kim, D. J. Lee, and I. L. Markov. 2012. SimPL: An effective placement algorithm. *TCAD* 31, 1 (Jan 2012), 50–60.
- [12] W. Li, S. Dhar, and D. Z. Pan. 2018. UTPlaceF: A routability-driven FPGA placer with physical and congestion aware packing. *TCAD* 37, 4 (April 2018), 869–882.
- [13] T. Lin, C. Chu, J. R. Shinnerl, I. Bustany, and I. Nedelchev. 2015. POLAR: A high performance mixed-size wirelength-driven placer with density constraints. *TCAD* 34, 3 (March 2015), 447–459.
- [14] P. Maidee, C. Ababei, and B. Bazargan. 2005. Timing-driven partitioning-based placement for island style FPGAs. *TCAD* 24, 3 (2005), 395–406.
- [15] R. Pattison, Z. Abuowaimar, S. Areibi, G. Grewal, and A. Vannelli. 2016. GPlace: A congestion-aware placement tool for UltraScale FPGAs. In *Proc. ICCAD*. 1–7.
- [16] C-W Pui, G. Chen, W-K. Chow, K-C. Lam, J. Kuang, P. Tu, H. Zhang, E. F. Y. Young, and B. Yu. 2016. RippleFPGA: A Routability-driven Placement for Large-scale Heterogeneous FPGAs. In *Proc. ICCAD*. 1–8.
- [17] Yang S., A. Gayasen, C. Mulpuri, S. Reddy, and R. Aggarwal. 2016. Routability-driven FPGA placement contest. In *Proc. ISPD*. 139–143.
- [18] Yang S., C. Mulpuri, S. Reddy, M. Kalase, S. Dasasathyan, M. E. Dehkordi, M. Tom, and R. Aggarwal. 2017. Clock-aware FPGA placement contest. In *Proc. ISPD*. 159–164.
- [19] P. Spindler and F. M. Schlichtmann, U. Johannes. 2008. Kraftwerk2: A fast force-directed quadratic placement approach using an accurate net model. *TCAD* 27, 8 (Aug 2008), 1398–1411.
- [20] D. Vercruyce, E. Vansteenkiste, and D. Stroobandt. 2018. Hierarchical force-based block spreading for analytic FPGA placement. In *Proc. FPL*.